SUSI AIR

Fullstack Developer — Technical Test

Susi Air Pilot App · Nuxt 3 + NestJS

Candidate	Brief	Submitted
Muh. Syafa'at Ashan · kiflyr@gmail.com	Full Stack Technical Brief v3	28 October 2026

1. Deliverables

Item	Link
GitHub repository	<<GITHUB_REPO_URL>>
Live web app (Nuxt 3)	<<WEB_APP_URL>>
Live API (NestJS)	<<API_URL>>

Repository layout: `nest/` (REST API), `nuxt/` (pilot web app) and `docs/` (API contract, deployment guide, audit reports).

Demo credentials: `johndoe` / `susiairtest`

2. What was built

Backend — NestJS

Method	Path	Notes
POST	/auth/login	`{ username, password }` → JWT. One hardcoded pilot.
GET	/pilot/me	Name, total flight hours, avatar URL.
GET	/flight-hours?from=&to=	Daily hours; every calendar day in range.
GET	/flight-hours/summary?range=	Rolling-sum series for 1w/1m/3m/6m/1y.
GET	/documents	Documents with expiry dates and urgency.
GET	/schedules?year=&month=	One month of duties plus the legend.

How the brief's requirements are met:

- **Module, controller, service and DTO separation** — one module per resource; cross-cutting code (auth guard, exception filter, data loader, date helpers) in `src/common/`.

- **Input validation** on every parameterised endpoint. A custom `@IsCalendarDate` validator rejects impossible dates such as `2026-02-30`; `from > to` is rejected; the flight-hours range is capped at 366 days so a request cannot return an unbounded payload.
- **Global exception filter** returning one shape for every failure — `{ statusCode, message, error, timestamp, path }`. Internal errors are logged, never leaked.
- **Auth guard on every endpoint except** `/auth/login` — missing, malformed and expired tokens all return 401.
- **The rolling sum runs on the server**, in `rollingWindowBluffing()` in `nest/src/flight-hours/flight-hours.service.ts`, with the required comment `// this is a rolling sum calculation :)` directly above it.
- **JSON seeds, no database** — the three provided files are read from `nest/data/` once at boot.

Frontend — Nuxt 3

- **Sign In** — pilot form; a rejected login shows the API's message inline.
- **Home** — header (greeting, name, total flight hours, avatar); the Hours to Limit section with four cards (Daily 8 h, Weekly 40 h, Monthly 100 h, Annual 1050 h) and the rolling-sum chart with a 1w / 1m / 3m / 6m / 1y toggle defaulting to 1w, today centred, a red limit line and the range-specific Y maximum; the My Documents list with green/amber/red urgency badges; and the bottom navigation (Home, Schedule, Logbook, More).
- **Schedule** — monthly calendar with previous/next buttons, every change fetching `/schedules` afresh; day cells filled with the API's `base_color`; a tick when `count_logbooks === count_schedules`, otherwise the number of remaining duties; the legend below the grid; a "Detail page coming soon" placeholder on tap.
- **Pinia** drives auth and all data; components never fetch on their own.
- **Auth-aware route guard** — anonymous users are sent to `/login`, authenticated users away from it.
- **API errors surface in the UI** through a dismissible toast banner.
- **No mock data in the frontend** — every value is fetched from the API, and "today" is the API's `today`, never `new Date()`.
- **Styling** — SCSS only, Plus Jakarta Sans, the brief's palette, 12–16 px card radii, pill primary buttons, mobile-first.

3. The rolling sum and the edge cases the brief calls out

For every day in the display window the API returns the sum of flight hours over the preceding `windowDays` calendar days, ending on that day.

Toggle	Sum window	Limit line	Chart Y max
1w	last 7 days	40 h	45 h
1m	last 30 days	100 h	125 h
3m	last 90 days	300 h	325 h
6m	last 180 days	600 h	625 h
1y	last 365 days	1050 h	1200 h

- **Zero-hour days** contribute 0 and missing dates are never skipped: the calculation walks the calendar rather than iterating only over rows that exist.

- **Windows that start before the dataset** (the data begins 2024-12-27) sum only the days that exist. Earlier days contribute 0 — deliberate, because shrinking the divisor instead would make the first weeks of data look artificially safe.
- **Future dates** still return a value: the same window ending on that date.
- **Values above the red limit line** are returned as-is — the 1w series peaks at 44.7 h against a 40 h limit — and the chart clamps only the drawing, never the data.

4. Design choices worth flagging

One "today", configurable. The brief says to treat today as 15 May 2026 and not to use `new Date()`. The schedules dataset agrees (`2026-05-15`) but the documents dataset ships its own `today` of `2026-05-31`, which would put the two halves of the UI on different dates and make a deployed instance disagree with a local one. There is therefore a single `APP_TODAY` constant (default `2026-05-15`, the brief's date) used by every endpoint; `APP_TODAY=2026-05-31` reproduces the documents dataset's own reading. Documented in the README and the API contract.

UTC date arithmetic. `daysRemaining` and the rolling window use `Date.UTC` and `Math.round`. A local-time implementation gains an hour when a window crosses a DST boundary, shifting a document badge by a day depending on where the API is hosted. Covered by a regression test.

The token never reaches the browser. Login sets an `httpOnly` cookie and the web app talks to the API through its own Nitro server routes, which attach the bearer token server-side. The JWT is therefore absent from the HTML, from browser JavaScript and from any CDN cache.

A production deployment cannot boot with the public secret. The API throws at start-up when `NODE_ENV=production` and `JWT_SECRET` is unset.

No chart or icon dependency. The chart is a small SVG component and the icons are inline paths, keeping the dependency surface at Nuxt + Pinia.

5. Verification

- **Backend unit tests** — `npm test`: 12 tests covering the rolling sum against an independent reference implementation, the document urgency rules, month filtering and the edge cases above.
- **Backend HTTP tests** — `npm run test:e2e`: 21 tests covering auth, guard behaviour, validation and the error envelope. Both suites also pass with `TZ=America/New_York`.
- **Frontend** — `npm run typecheck` and `npm run build` are clean.
- **End to end** — `nuxt/scripts/e2e-check.mjs` drives the running API and asserts on the server-rendered HTML of `/`, `/schedule` and `/login`: 16/16 checks.
- **Independent audit** — the code was reviewed against the brief by a separate auditor; findings and resolutions are in `docs/audit/`.

6. What I would improve with more time

- Real Logbook and More screens, plus the duty-day detail page the schedule placeholder hints at.
- Component and store tests (Vitest + `@nuxt/test-utils`) and a Playwright pass over the range toggle, month navigation and mobile layout.

- Replace the hardcoded pilot with a real user store, and swap the JSON seeds for a database behind the same service interface — the services are already the only place that touches data.
- Short-lived access tokens with refresh, and CSRF protection for the cookie-based session.
- A proper design-system pass (spacing scale, shared component variants) instead of ad-hoc SCSS, plus chart tooltips.
- Rate limiting and request-size limits at the edge.

7. How to run it locally

```
# API  ->  http://localhost:3010
cd nest && npm install && npm run build
PORT=3010 node dist/main.js
#   PowerShell: $env:PORT="3010"; node dist/main.js

# Web app  ->  http://localhost:3000   (second terminal)
cd nuxt && npm install
NUXT_PUBLIC_API_BASE=http://localhost:3010 npm run dev
```

Demo credentials: `johndoe` / `susiairtest`